

```

var fileOpenPicker = new FileOpenPicker();
fileOpenPicker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;
fileOpenPicker.FileTypeFilter.Add(".png");
fileOpenPicker.ViewMode = PickerViewMode.Thumbnail;

var inputFiles = await fileOpenPicker.PickMultipleFilesAsync();

if (inputFiles == null)
{
    // The user cancelled the picking operation
    return null;
}
if(inputFiles.Count >= maxFrames)
{
    Debug.WriteLine("You can only choose up to {0} image(s) to input.",
maxFrames);
}

```

Now that we have the correct number of frames selected, we need to decode the frames into [SoftwareBitmaps](#) and ensure that the SoftwareBitmaps are in the correct format for LowLightFusion.

C#

```

SoftwareBitmap softwareBitmap;

// Decode the images into bitmaps
for (int i = 0; i < inputFiles.Count; i++)
{
    using (IRandomAccessStream stream = await
inputFiles[0].OpenAsync(FileAccessMode.Read))
    {
        // Create the decoder from the stream
        BitmapDecoder decoder = await BitmapDecoder.CreateAsync(stream);

        // Get the SoftwareBitmap representation of the file
        softwareBitmap = await decoder.GetSoftwareBitmapAsync();
        framelist.Add(softwareBitmap);
    }
}

// Ensure that the selected bitmap frames have an acceptable pixel format
for Low Light Fusion.
// For this sample, we'll use the pixel format at index 0.
IReadOnlyList<BitmapPixelFormat> formats =
LowLightFusion.SupportedBitmapPixelFormats;
BitmapPixelFormat llfFormat = formats[0];
for (int i = 0; i < framelist.Count; i++)
{
    if (framelist[i].BitmapPixelFormat == llfFormat)
    {
        // The pixel format is acceptable
    }
}

```

```

    }
    else
    {
        // Convert the pixel format
        framelist[i] = SoftwareBitmap.Convert(framelist[i], llfFormat);
    }
}
return framelist;

```

Fuse the bitmaps into a single bitmap

Now that we have a correct number of frames in an acceptable format, we can use the [FuseAsync](#) method to apply the Low Light Fusion algorithm. Our result will be the processed image, with improved clarity, in the form of a SoftwareBitmap.

C#

```
LowLightFusionResult result = await LowLightFusion.FuseAsync(pictures);
```

Finally, we'll clean up the resulting SoftwareBitmap by encoding and saving it into a user friendly, "regular" image, similar to the input images that we started with.

C#

```

// Convert bitmap to Rgba16, Rgba8, or Bgra8 for encoding.
var frameConverted = SoftwareBitmap.Convert(frame, BitmapPixelFormat.Bgra8);

var fileSavePicker = new FileSavePicker();
fileSavePicker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;
fileSavePicker.FileTypeChoices.Add("PNG files", new List<string>() { ".png" });
fileSavePicker.SuggestedFileName = "LLF Output";

var outputFile = await fileSavePicker.PickSaveFileAsync();
if (outputFile == null)
{
    // The user cancelled the picking operation
    return;
}

// Encode and save the image
using (IRandomAccessStream stream = await
outputFile.OpenAsync(FileAccessMode.ReadWrite))
{
    // Create an encoder with the desired format
    BitmapEncoder encoder = await
BitmapEncoder.CreateAsync(BitmapEncoder.JpegEncoderId, stream);

```

```

// Set the software bitmap
encoder.SetSoftwareBitmap(frameConverted);

try
{
    await encoder.FlushAsync();
}
catch (Exception err)
{
    switch (err.HResult)
    {
        case unchecked((int)0x88982F81): //
WINCODEC_ERR_UNSUPPORTEDOPERATION // If the encoder does not
support writing a thumbnail, then try again // but disable thumbnail
generation.
            encoder.IsThumbnailGenerated = false;
            break;
        default:
            throw err;
    }
}

if (encoder.IsThumbnailGenerated == false)
{
    await encoder.FlushAsync();
}
}

```

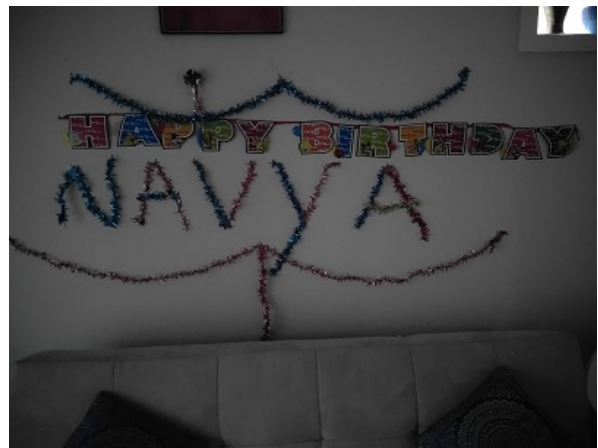
Before and after

Here's an example of an input image and the resulting output image after applying the Low Light Fusion algorithm.

Input Frame



Low Light Fusion Output



You can see from the input frame that the lighting and the clarity of the shadows surrounding the banner have been improved.

Related topics

[LowLightFusion Class](#)

[LowLightFusionResult Class](#)

BitmapEncoder options reference

Article • 10/20/2022

This article lists the encoding options that can be used with [BitmapEncoder](#). An encoding option is defined by its name, which is a string, and a value in a particular data type ([Windows.Foundation.PropertyType](#)). For information about working with images, see [Create, edit, and save bitmap images](#).

Name	PropertyType	Usage notes	Valid formats
ImageQuality	single	Valid values from 0 to 1.0. Higher values indicate higher quality	JPEG, JPEG-XR
CompressionQuality	single	Valid values from 0 to 1.0. Higher values indicate a more efficient and slower compression scheme	TIFF
Lossless	boolean	If this is set to true, the ImageQuality option is ignored	JPEG-XR
InterlaceOption	boolean	Whether to interlace the image	PNG
FilterOption	uint8	Use the PngFilterMode enumeration	PNG
TiffCompressionMethod	uint8	Use the TiffCompressionMode enumeration	TIFF
Luminance	uint32Array	An array of 64 elements containing luminance quantization constants	JPEG
Chrominance	uint32Array	An array of 64 elements containing chrominance quantization constants	JPEG
JpegYCrCbSubsampling	uint8	Use the JpegSubsamplingMode enumeration	JPEG
SuppressApp0	boolean	Whether to suppress the creation of an App0 metadata block	JPEG
EnableV5Header32bppBGRA	boolean	Whether to encode to a version 5 BMP which supports alpha	BMP

Related topics

- Create, edit, and save bitmap images
- Supported codecs

Image Metadata

Article • 10/20/2022

This article shows how to read and write image metadata properties and how to geotag files using the [GeotagHelper](#) utility class.

Image properties

The [StorageFile.Properties](#) property returns a [StorageItemContentProperties](#) object that provides access to content-related information about the file. Get the image-specific properties by calling [GetImagePropertiesAsync](#). The returned [ImageProperties](#) object exposes members that contain basic image metadata fields, like the title of the image and the capture date.

C#

```
private async void GetImageProperties(StorageFile imageFile)
{
    ImageProperties props = await
imageFile.Properties.GetImagePropertiesAsync();

    string title = props.Title;
    if (title == null)
    {
        // Format does not support, or image does not contain Title property
    }

    DateTimeOffset date = props.DateTaken;
    if (date == null)
    {
        // Format does not support, or image does not contain DateTaken
property
    }
}
```

To access a larger set of file metadata, use the Windows Property System, a set of file metadata properties that can be retrieved with a unique string identifier. Create a list of strings and add the identifier for each property you want to retrieve. The [ImageProperties.RetrievePropertiesAsync](#) method takes this list of strings and returns a dictionary of key/value pairs where the key is the property identifier and the value is the property value.

C#

```

ImageProperties props = await
imageFile.Properties.GetImagePropertiesAsync();

var requests = new System.Collections.Generic.List<string>();
requests.Add("System.Photo.Orientation");
requests.Add("System.Photo.Aperture");

IDictionary<string, object> retrievedProps = await
props.RetrievePropertiesAsync(requests);

ushort orientation;
if (retrievedProps.ContainsKey("System.Photo.Orientation"))
{
    orientation = (ushort)retrievedProps["System.Photo.Orientation"];
}

double aperture;
if (retrievedProps.ContainsKey("System.Photo.Aperture"))
{
    aperture = (double)retrievedProps["System.Photo.Aperture"];
}

```

- For a complete list of Windows Properties, including the identifiers and type for each property, see [Windows Properties](#).
- Some properties are only supported for certain file containers and image codecs. For a listing of the image metadata supported for each image type, see [Photo Metadata Policies](#).
- Because properties that are unsupported may return a null value when retrieved, always check for null before using a returned metadata value.

Geotag helper

GeotagHelper is a utility class that makes it easy to tag images with geographic data using the [Windows.Devices.Geolocation](#) APIs directly, without having to manually parse or construct the metadata format.

If you already have a [Geopoint](#) object representing the location you want to tag in the image, either from a previous use of the geolocation APIs or some other source, you can set the geotag data by calling [GeotagHelper.SetGeotagAsync](#) and passing in a [StorageFile](#) and the [Geopoint](#).

C#

```

var point = new Geopoint(
new BasicGeoposition

```



```
{
    Latitude = 48.8567,
    Longitude = 2.3508,
});

await GeotagHelper.SetGeotagAsync(imageFile, point);
```

To set the geotag data using the device's current location, create a new [Geolocator](#) object and call [GeotagHelper.SetGeotagFromGeolocatorAsync](#) passing in the [Geolocator](#) and the file to be tagged.

```
C#

var locator = new Geolocator();

// Shows the user consent UI if needed
var accessStatus = await Geolocator.RequestAccessAsync();
if (accessStatus == GeolocationAccessStatus.Allowed)
{
    await GeotagHelper.SetGeotagFromGeolocatorAsync(imageFile, locator);
}
```

- You must include the **location** device capability in your app manifest in order to use the [SetGeotagFromGeolocatorAsync](#) API.
- You must call [RequestAccessAsync](#) before calling [SetGeotagFromGeolocatorAsync](#) to ensure the user has granted your app permission to use their location.
- For more information on the geolocation APIs, see [Maps and location](#).

To get a [GeoPoint](#) representing the geotagged location of an image file, call [GetGeotagAsync](#).

```
C#

Geopoint geoPoint = await GeotagHelper.GetGeotagAsync(imageFile);
```

Decode and encode image metadata

The most advanced way of working with image data is to read and write the properties on the stream level using a [BitmapDecoder](#) or a [BitmapEncoder](#). For these operations you can use Windows Properties to specify the data you are reading or writing, but you can also use the metadata query language provided by the Windows Imaging Component (WIC) to specify the path to a requested property.

Reading image metadata using this technique requires you to have a [BitmapDecoder](#) that was created with the source image file stream. For information on how to do this, see [Imaging](#).

Once you have the decoder, create a list of strings and add a new entry for each metadata property you want to retrieve, using either the Windows Property identifier string or a WIC metadata query. Call the [BitmapPropertiesView.GetPropertiesAsync](#) method on the decoder's [BitmapProperties](#) member to request the specified properties. The properties are returned in a dictionary of key/value pairs containing the property name or path and the property value.

C#

```
private async void ReadImageMetadata(BitmapDecoder bitmapDecoder)
{
    var requests = new System.Collections.Generic.List<string>();
    requests.Add("System.Photo.Orientation"); // Windows property key for EXIF orientation
    requests.Add("/xmp/dc:creator"); // WIC metadata query for Dublin Core creator

    try
    {
        var retrievedProps = await
        bitmapDecoder.BitmapProperties.GetPropertiesAsync(requests);

        ushort orientation;
        if (retrievedProps.ContainsKey("System.Photo.Orientation"))
        {
            orientation =
            (ushort)retrievedProps["System.Photo.Orientation"].Value;
        }

        string creator;
        if (retrievedProps.ContainsKey("/xmp/dc:creator"))
        {
            creator = (string)retrievedProps["/xmp/dc:creator"].Value;
        }
    }
    catch (Exception err)
    {
        switch (err.HResult)
        {
            case unchecked((int)0x88982F41): //
            WINCODEC_ERR_PROPERTYNOTSUPPORTED // The file format does not
            support the requested metadata.
                break;
            case unchecked((int)0x88982F81): //
            WINCODEC_ERR_UNSUPPORTEDOPERATION
        }
    }
}
```

```

// The file format does not
support any metadata.
    default:
        throw err;
    }
}
}

```

- For information on the WIC metadata query language and the properties supported, see [WIC image format native metadata queries](#).
- Many metadata properties are only supported by a subset of image types. [GetPropertiesAsync](#) will fail with the error code 0x88982F41 if one of the requested properties is not supported by the image associated with the decoder and 0x88982F81 if the image does not support metadata at all. The constants associated with these error codes are WINCODEC_ERR_PROPERTYNOTSUPPORTED and WINCODEC_ERR_UNSUPPORTEDOPERATION and are defined in the winerror.h header file.
- Because an image may or may not contain a value for a particular property, use the [IDictionary.ContainsKey](#) to verify that a property is present in the results before attempting to access it.

Writing image metadata to the stream requires a **BitmapEncoder** associated with the image output file.

Create a [BitmapPropertySet](#) object to contain the property values you want set. Create a [BitmapTypedValue](#) object to represent the property value. This object uses an **object** as the value and member of the [PropertyType](#) enumeration that defines the type of the value. Add the [BitmapTypedValue](#) to the [BitmapPropertySet](#) and then call [BitmapProperties.SetPropertiesAsync](#) to cause the encoder to write the properties to the stream.

C#

```

private async void WriteImageMetadata(BitmapEncoder bitmapEncoder)
{
    var propertySet = new Windows.Graphics.Imaging.BitmapPropertySet();
    var orientationValue = new Windows.Graphics.Imaging.BitmapTypedValue(
        1, // Defined as EXIF orientation = "normal"
        Windows.Foundation.PropertyType.UInt16
    );

    propertySet.Add("System.Photo.Orientation", orientationValue);

    try
    {

```

```

        await
        bitmapEncoder.BitmapProperties.SetPropertiesAsync(propertySet);
    }
    catch (Exception err)
    {
        switch (err.HResult)
        {
            case unchecked((int)0x88982F41): //
WINCODEC_ERR_PROPERTYNOTSUPPORTED           // The file format does not
support this property.
                break;
            default:
                throw err;
        }
    }
}

```

- For details on which properties are supported for which image file types, see [Windows Properties](#), [Photo Metadata Policies](#), and [WIC image format native metadata queries](#).
- [SetPropertiesAsync](#) will fail with the error code 0x88982F41 if one of the requested properties is not supported by the image associated with the encoder.

Related topics

- [Imaging](#)

Transcode media files

Article • 10/20/2022

You can use the [Windows.Media.Transcoding](#) APIs to transcode video files from one format to another.

Transcoding is the conversion of a digital media file, such as a video or audio file, from one format to another. This is usually done by decoding and then re-encoding the file. For example, you might convert a Windows Media file to MP4 so that it can be played on a portable device that supports MP4 format. Or, you might convert a high-definition video file to a lower resolution. In that case, the re-encoded file might use the same codec as the original file, but it would have a different encoding profile.

Set up your project for transcoding

In addition to the namespaces referenced by the default project template, you will need to reference these namespaces in order to transcode media files using the code in this article.

C#

```
using Windows.Storage;
using Windows.Media.MediaProperties;
using Windows.Media.Transcoding;
```

Select source and destination files

The way that your app determines the source and destination files for transcoding depends on your implementation. This example uses a [FileOpenPicker](#) and a [FileSavePicker](#) to allow the user to pick a source and a destination file.

C#

```
var openPicker = new Windows.Storage.Pickers.FileOpenPicker();

openPicker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.VideosLibrary;
openPicker.FileTypeFilter.Add(".wmv");
openPicker.FileTypeFilter.Add(".mp4");

StorageFile source = await openPicker.PickSingleFileAsync();

var savePicker = new Windows.Storage.Pickers.FileSavePicker();
```

```

savePicker.SuggestedStartLocation =
    Windows.Storage.Pickers.PickerLocationId.VideosLibrary;

savePicker.DefaultFileExtension = ".mp4";
savePicker.SuggestedFileName = "New Video";

savePicker.FileTypeChoices.Add("MPEG4", new string[] { ".mp4" });

StorageFile destination = await savePicker.PickSaveFileAsync();

```

Create a media encoding profile

The encoding profile contains the settings that determine how the destination file will be encoded. This is where you have the greatest number of options when you transcode a file.

The [MediaEncodingProfile](#) class provides static methods for creating predefined encoding profiles:

Methods for creating Audio-only encoding profiles

Method	Profile
CreateAlac	Apple Lossless Audio Codec (ALAC) audio
CreateFlac	Free Lossless Audio Codec (FLAC) audio.
CreateM4a	AAC audio (M4A)
CreateMp3	MP3 audio
CreateWav	WAV audio
CreateWmv	Windows Media Audio (WMA)

Methods for creating audio / video encoding profiles

Method	Profile
CreateAvi	AVI
CreateHevc	High Efficiency Video Coding (HEVC) video, also known as H.265 video
CreateMp4	MP4 video (H.264 video plus AAC audio)

Method	Profile
CreateWmv	Windows Media Video (WMV)

The following code creates a profile for MP4 video.

C#

```
MediaEncodingProfile profile =  
    MediaEncodingProfile.CreateMp4(VideoEncodingQuality.HD720p);
```

The static [CreateMp4](#) method creates an MP4 encoding profile. The parameter for this method gives the target resolution for the video. In this case, [VideoEncodingQuality.hd720p](#) means 1280 x 720 pixels at 30 frames per second. ("720p" stands for 720 progressive scan lines per frame.) The other methods for creating predefined profiles all follow this pattern.

Alternatively, you can create a profile that matches an existing media file by using the [MediaEncodingProfile.CreateFromFileAsync](#) method. Or, if you know the exact encoding settings that you want, you can create a new [MediaEncodingProfile](#) object and fill in the profile details.

Transcode the file

To transcode the file, create a new [MediaTranscoder](#) object and call the [MediaTranscoder.PrepareFileTranscodeAsync](#) method. Pass in the source file, the destination file, and the encoding profile. Then call the [TranscodeAsync](#) method on the [PrepareTranscodeResult](#) object that was returned from the async transcode operation.

C#

```
MediaTranscoder transcoder = new MediaTranscoder();  
  
PrepareTranscodeResult prepareOp = await  
    transcoder.PrepareFileTranscodeAsync(source, destination, profile);  
  
if (prepareOp.CanTranscode)  
{  
    var transcodeOp = prepareOp.TranscodeAsync();  
  
    transcodeOp.Progress +=  
        new AsyncActionProgressHandler<double>(TranscodeProgress);  
    transcodeOp.Completed +=  
        new AsyncActionWithProgressCompletedHandler<double>  
            (TranscodeComplete);  
}
```

```

else
{
    switch (prepareOp.FailureReason)
    {
        case TranscodeFailureReason.CodecNotFound:
            System.Diagnostics.Debug.WriteLine("Codec not found.");
            break;
        case TranscodeFailureReason.InvalidProfile:
            System.Diagnostics.Debug.WriteLine("Invalid profile.");
            break;
        default:
            System.Diagnostics.Debug.WriteLine("Unknown failure.");
            break;
    }
}

```

Respond to transcoding progress

You can register events to respond when the progress of the asynchronous [TranscodeAsync](#) changes. These events are part of the async programming framework for Universal Windows Platform (UWP) apps and are not specific to the transcoding API.

C#

```

void TranscodeProgress(IAsyncActionWithProgress<double> asyncInfo, double
percent)
{
    // Display or handle progress info.
}

void TranscodeComplete(IAsyncActionWithProgress<double> asyncInfo,
AsyncStatus status)
{
    asyncInfo.GetResults();
    if (asyncInfo.Status == AsyncStatus.Completed)
    {
        // Display or handle complete info.
    }
    else if (asyncInfo.Status == AsyncStatus.Canceled)
    {
        // Display or handle cancel info.
    }
    else
    {
        // Display or handle error info.
    }
}

```


Process media files in the background

Article • 10/20/2022

This article shows you how to use the [MediaProcessingTrigger](#) and a background task to process media files in the background.

The example app described in this article allows the user to select an input media file to transcode and specify an output file for the transcoding result. Then, a background task is launched to perform the transcoding operation. The [MediaProcessingTrigger](#) is intended to support many different media processing scenarios besides transcoding, including rendering media compositions to disk and uploading processed media files after processing is complete.

For more detailed information on the different Universal Windows app features utilized in this sample, see:

- [Transcode media files](#)
- [Launching resuming and background tasks](#)
- [Tiles badges and notifications](#)

Create a media processing background task

To add a background task to your existing solution in Microsoft Visual Studio, Enter a name for your comp

1. From the **File** menu, select **Add** and then **New Project....**
2. Select the project type **Windows Runtime Component (Universal Windows)**.
3. Enter a name for your new component project. This example uses the project name **MediaProcessingBackgroundTask**.
4. Click OK.

In **Solution Explorer**, right-click the icon for the "Class1.cs" file that is created by default and select **Rename**. Rename the file to "MediaProcessingTask.cs". When Visual Studio asks if you want to rename all of the references to this class, click **Yes**.

In the renamed class file, add the following **using** directives to include these namespaces in your project.

C#

```
using Windows.ApplicationModel.Background;  
using Windows.Storage;  
using Windows.UI.Notifications;
```

```
using Windows.Data.Xml.Dom;
using Windows.Media.MediaProperties;
using Windows.Media.Transcoding;
using System.Threading;
```

Update your class declaration to make your class inherit from **IBackgroundTask**.

C#

```
public sealed class MediaProcessingTask : IBackgroundTask
{
```

Add the following member variables to your class:

- An **IBackgroundTaskInstance** that will be used to update the foreground app with the progress of the background task.
- A **BackgroundTaskDeferral** that keeps the system from shutting down your background task while media transcoding is being performed asynchronously.
- A **CancellationTokenSource** object that can be used to cancel the asynchronous transcoding operation.
- The **MediaTranscoder** object that will be used to transcode media files.

C#

```
IBackgroundTaskInstance backgroundTaskInstance;
BackgroundTaskDeferral deferral;
CancellationTokenSource cancellationTokenSource = new CancellationTokenSource();
MediaTranscoder transcoder;
```

The system calls **Run** method of a background task when the task is launched. Set the **IBackgroundTask** object passed into the method to the corresponding member variable. Register a handler for the **Canceled** event, which will be raised if the system needs to shut down the background task. Then, set the **Progress** property to zero.

Next, call the background task object's **GetDeferral** method to obtain a deferral. This tells the system not to shut down your task because you are performing asynchronous operations.

Next, call the helper method **TranscodeFileAsync**, which is defined in the next section. If that completes successfully, a helper method is called to launch a toast notification to alert the user that transcoding is complete.

At the end of the **Run** method, call **Complete** on the deferral object to let the system know that your background task is complete and can be terminated.

C#

```
public async void Run(IBackgroundTaskInstance taskInstance)
{
    Debug.WriteLine("In background task Run method");

    backgroundTaskInstance = taskInstance;
    taskInstance.Canceled += new
BackgroundTaskCanceledEventHandler(OnCanceled);
    taskInstance.Progress = 0;

    deferral = taskInstance.GetDeferral();
    Debug.WriteLine("Background " + taskInstance.Task.Name + " is called @ "
+ (DateTime.Now).ToString());

    try
    {
        await TranscodeFileAsync();
        ApplicationData.Current.LocalSettings.Values["TranscodingStatus"] =
"Completed Successfully";
        SendToastNotification("File transcoding complete.");
    }
    catch (Exception e)
    {
        Debug.WriteLine("Exception type: {0}", e.ToString());
        ApplicationData.Current.LocalSettings.Values["TranscodingStatus"] =
"Error occurred: " + e.ToString();
    }

    deferral.Complete();
}
```

In the **TranscodeFileAsync** helper method, the file names for the input and output files for the transcoding operations are retrieved from the **LocalSettings** for your app. These values will be set by your foreground app. Create a **StorageFile** object for the input and output files and then create an encoding profile to use for transcoding.

Call **PrepareFileTranscodeAsync**, passing in the input file, output file, and encoding profile. The **PrepareTranscodeResult** object returned from this call lets you know if transcoding can be performed. If the **CanTranscode** property is true, call **TranscodeAsync** to perform the transcoding operation.

The **AsTask** method enables you to track the progress the asynchronous operation or cancel it. Create a new **Progress** object, specifying the units of progress you desire and the name of the method that will be called to notify you of the current progress of the task. Pass the **Progress** object into the **AsTask** method along with the cancellation token that allows you to cancel the task.

C#

```
private async Task TranscodeFileAsync()
{
    transcoder = new MediaTranscoder();

    try
    {
        var settings = ApplicationData.Current.LocalSettings;

        settings.Values["TranscodingStatus"] = "Started";

        var inputFileName =
ApplicationData.Current.LocalSettings.Values["InputFileName"] as string;
        var outputFileName =
ApplicationData.Current.LocalSettings.Values["OutputFileName"] as string;

        if (inputFileName == null || outputFileName == null)
        {
            return;
        }

        // retrieve the transcoding information
        var inputFile = await
Windows.Storage.StorageFile.GetFileFromPathAsync(inputFileName);
        var outputFile = await
Windows.Storage.StorageFile.GetFileFromPathAsync(outputFileName);

        // create video encoding profile
        MediaEncodingProfile encodingProfile =
MediaEncodingProfile.CreateMp4(VideoEncodingQuality.HD720p);

        Debug.WriteLine("PrepareFileTranscodeAsync");
        settings.Values["TranscodingStatus"] = "Preparing to transcode ";
        PrepareTranscodeResult preparedTranscodeResult = await
transcoder.PrepareFileTranscodeAsync(
            inputFile,
            outputFile,
            encodingProfile);

        if (preparedTranscodeResult.CanTranscode)
        {
            var startTime =
TimeSpan.FromMilliseconds(DateTime.Now.Millisecond);
            Debug.WriteLine("Starting transcoding @" + startTime);

            var progress = new Progress<double>(TranscodeProgress);
            settings.Values["TranscodingStatus"] = "Transcoding ";
            settings.Values["ProcessingFileName"] = inputFileName;
            await
preparedTranscodeResult.TranscodeAsync().AsTask(cancelTokenSource.Token,
progress);
        }
    }
}
```

```

    }
    else
    {
        Debug.WriteLine("Source content could not be transcoded.");
        Debug.WriteLine("Transcode status: " +
preparedTranscodeResult.FailureReason.ToString());
        var endTime =
        TimeSpan.FromMilliseconds(DateTime.Now.Millisecond);
        Debug.WriteLine("End time = " + endTime);
    }
}
catch (Exception e)
{
    Debug.WriteLine("Exception type: {0}", e.ToString());
    throw;
}
}

```

In the method you used to create the Progress object in the previous step, **Progress**, set the progress of the background task instance. This will pass the progress to the foreground app, if it is running.

C#

```

void TranscodeProgress(double percent)
{
    Debug.WriteLine("Transcoding progress: " +
percent.ToString().Split('.')[0] + "%");
    backgroundTaskInstance.Progress = (uint)percent;
}

```

The **SendToastNotification** helper method creates a new toast notification by getting a template XML document for a toast that only has text content. The text element of the toast XML is set and then a new **ToastNotification** object is created from the XML document. Finally, the toast is shown to the user by calling **ToastNotifier.Show**.

C#

```

private void SendToastNotification(string toastMessage)
{
    ToastTemplateType toastTemplate = ToastTemplateType.ToastText01;
    XmlDocument toastXml =
    ToastNotificationManager.GetTemplateContent(toastTemplate);

    //Supply text content for your notification
    XmlNodeList toastTextElements = toastXml.GetElementsByTagName("text");
    toastTextElements[0].AppendChild(toastXml.CreateTextNode(toastMessage));

    //Create the toast notification based on the XML content you've
    specified.
}

```

```
ToastNotification toast = new ToastNotification(toastXml);

//Send your toast notification.
ToastNotificationManager.CreateToastNotifier().Show(toast);
}
```

In the handler for the **Canceled** event, which is called when the system cancels your background task, you can log the error for telemetry purposes.

C#

```
private void OnCanceled(IBackgroundTaskInstance sender,
BackgroundTaskCancellationReason reason)
{
    Debug.WriteLine("Background " + sender.Task.Name + " Cancel
Requested..." + reason.ToString());
}
```

Register and launch the background task

Before you can launch the background task from your foreground app, you must update your foreground app's Package.appmanifest file to let the system know that your app uses a background task.

1. In **Solution Explorer**, double-click the Package.appmanifest file icon to open the manifest editor.
2. Select the **Declarations** tab.
3. From **Available Declarations**, select **Background Tasks** and click **Add**.
4. Under **Supported Declarations** make sure that the **Background Tasks** item is selected. Under **Properties**, select the checkbox for **Media processing**.
5. In the **Entry Point** text box, specify the namespace and class name for your background test, separated by a period. For this example, the entry is:

C#

```
MediaProcessingBackgroundTask.MediaProcessingTask
```

Next, you need to add a reference to your background task to your foreground app.

1. In **Solution Explorer**, under your foreground app project, right-click the **References** folder and select **Add Reference....**
2. Expand the **Projects** node and select **Solution**.
3. Check the box next to your background task project and click **OK**.

The rest of the code in this example should be added to your foreground app. First, you will need to add the following namespaces to your project.

C#

```
using Windows.ApplicationModel.Background;  
using Windows.Storage;
```

Next, add the following member variables that are needed to register the background task.

C#

```
MediaProcessingTrigger mediaProcessingTrigger;  
string backgroundTaskBuilderName = "TranscodingBackgroundTask";  
BackgroundTaskRegistration taskRegistration;
```

The **PickFilesToTranscode** helper method uses a [FileOpenPicker](#) and a [FileSavePicker](#) to open the input and output files for transcoding. The user may select files in a location that your app does not have access to. To make sure your background task can open the files, add them to the [FutureAccessList](#) for your app.

Finally, set entries for the input and output file names in the [LocalSettings](#) for your app. The background task retrieves the file names from this location.

C#

```
private async void PickFilesToTranscode()  
{  
    var openPicker = new Windows.Storage.Pickers.FileOpenPicker();  
  
    openPicker.SuggestedStartLocation =  
        Windows.Storage.Pickers.PickerLocationId.VideosLibrary;  
    openPicker.FileTypeFilter.Add(".wmv");  
    openPicker.FileTypeFilter.Add(".mp4");  
  
    StorageFile source = await openPicker.PickSingleFileAsync();  
  
    var savePicker = new Windows.Storage.Pickers.FileSavePicker();  
  
    savePicker.SuggestedStartLocation =  
        Windows.Storage.Pickers.PickerLocationId.VideosLibrary;  
  
    savePicker.DefaultFileExtension = ".mp4";  
    savePicker.SuggestedFileName = "New Video";  
  
    savePicker.FileTypeChoices.Add("MPEG4", new string[] { ".mp4" });  
  
    StorageFile destination = await savePicker.PickSaveFileAsync();
```

```

        if(source == null || destination == null)
        {
            return;
        }

        var storageItemAccessList =
Windows.Storage.AccessCache.StorageApplicationPermissions.FutureAccessList;
        storageItemAccessList.Add(source);
        storageItemAccessList.Add(destination);

        ApplicationData.Current.LocalSettings.Values["InputFileName"] =
source.Path;
        ApplicationData.Current.LocalSettings.Values["OutputFileName"] =
destination.Path;
    }

```

To register the background task, create a new [MediaProcessingTrigger](#) and a new [BackgroundTaskBuilder](#). Set the name of the background task builder so that you can identify it later. Set the [TaskEntryPoint](#) to the same namespace and class name string you used in the manifest file. Set the [Trigger](#) property to the [MediaProcessingTrigger](#) instance.

Before registering the task, make sure you unregister any previously registered tasks by looping through the [AllTasks](#) collection and calling [Unregister](#) on any tasks that have the name you specified in the [BackgroundTaskBuilder.Name](#) property.

Register the background task by calling [Register](#). Register handlers for the [Completed](#) and [Progress](#) events.

C#

```

private void RegisterBackgroundTask()
{
    // New a MediaProcessingTrigger
    mediaProcessingTrigger = new MediaProcessingTrigger();

    var builder = new BackgroundTaskBuilder();

    builder.Name = backgroundTaskBuilderName;
    builder.TaskEntryPoint =
"MediaProcessingBackgroundTask.MediaProcessingTask";
    builder.SetTrigger(mediaProcessingTrigger);

    // unregister old ones
    foreach (var cur in BackgroundTaskRegistration.AllTasks)
    {
        if (cur.Value.Name == backgroundTaskBuilderName)
        {
            cur.Value.Unregister(true);
        }
    }
}

```



```

    }
}

taskRegistration = builder.Register();
taskRegistration.Progress += new
BackgroundTaskProgressEventHandler(OnProgress);
taskRegistration.Completed += new
BackgroundTaskCompletedEventHandler(OnCompleted);

return;
}

```

A typical app will register their background task when the app is initially launched, such as in the **OnNavigatedTo** event.

Launch the background task by calling the **MediaProcessingTrigger** object's **RequestAsync** method. The **MediaProcessingTriggerResult** object returned by this method lets you know whether the background task was started successfully, and if not, lets you know why the background task wasn't launched.

C#

```

private async void LaunchBackgroundTask()
{
    var success = true;

    if (mediaProcessingTrigger != null)
    {
        MediaProcessingTriggerResult activationResult;
        activationResult = await mediaProcessingTrigger.RequestAsync();

        switch (activationResult)
        {
            case MediaProcessingTriggerResult.Allowed:
                // Task starting successfully
                break;

            case MediaProcessingTriggerResult.CurrentlyRunning:
                // Already Triggered

            case MediaProcessingTriggerResult.DisabledByPolicy:
                // Disabled by system policy

            case MediaProcessingTriggerResult.UnknownError:
                // All other failures
                success = false;
                break;
        }

        if (!success)
        {
            // Unregister the media processing trigger background task

```

```
        taskRegistration.Unregister(true);
    }
}
```

A typical app will launch the background task in response to user interaction, such as in the **Click** event of a UI control.

The **OnProgress** event handler is called when the background task updates the progress of the operation. You can use this opportunity to update your UI with progress information.

C#

```
private void OnProgress(IBackgroundTaskRegistration task,
    BackgroundTaskProgressEventArgs args)
{
    string progress = "Progress: " + args.Progress + "%";
    Debug.WriteLine(progress);
}
```

The **OnCompleted** event handler is called when the background task has finished running. This is another opportunity to update your UI to give status information to the user.

C#

```
private void OnCompleted(IBackgroundTaskRegistration task,
    BackgroundTaskCompletedEventArgs args)
{
    Debug.WriteLine(" background task complete");
}
```

Audio graphs

Article • 10/20/2022

This article shows how to use the APIs in the [Windows.Media.Audio](#) namespace to create audio graphs for audio routing, mixing, and processing scenarios.

An *audio graph* is a set of interconnected audio nodes through which audio data flows.

- *Audio input nodes* supply audio data to the graph from audio input devices, audio files, or from custom code. lat
- *Audio output nodes* are the destination for audio processed by the graph. Audio can be routed out of the graph to audio output devices, audio files, or custom code.
- *Submix nodes* take audio from one or more nodes and combine them into a single output that can be routed to other nodes in the graph.

After all of the nodes have been created and the connections between them set up, you simply start the audio graph and the audio data flows from the input nodes, through any submix nodes, to the output nodes. This model makes scenarios such as recording from a device's microphone to an audio file, playing audio from a file to a device's speaker, or mixing audio from multiple sources quick and easy to implement.

Additional scenarios are enabled with the addition of audio effects to the audio graph. Every node in an audio graph can be populated with zero or more audio effects that perform audio processing on the audio passing through the node. There are several built-in effects such as echo, equalizer, limiting, and reverb that can be attached to an audio node with just a few lines of code. You can also create your own custom audio effects that work exactly the same as the built-in effects.

ⓘ Note

The [AudioGraph UWP sample](#) [↗] implements the code discussed in this overview. You can download the sample to see the code in context or to use as a starting point for your own app.

Choosing Windows Runtime AudioGraph or XAudio2

The Windows Runtime audio graph APIs offer functionality that can also be implemented by using the COM-based [XAudio2 APIs](#). The following are features of the Windows Runtime audio graph framework that differ from XAudio2.

The Windows Runtime audio graph APIs:

- Are significantly easier to use than XAudio2.
- Can be used from C# in addition to being supported for C++.
- Can use audio files, including compressed file formats, directly. XAudio2 only operates on audio buffers and does not provide any file I/O capabilities.
- Can use the low-latency audio pipeline in Windows 10.
- Support automatic endpoint switching when default endpoint parameters are used. For example, if the user switches from a device's speaker to a headset, the audio is automatically redirected to the new input.

AudioGraph class

The [AudioGraph](#) class is the parent of all nodes that make up the graph. Use this object to create instances of all of the audio node types. Create an instance of the [AudioGraph](#) class by initializing an [AudioGraphSettings](#) object containing configuration settings for the graph, and then calling [AudioGraph.CreateAsync](#). The returned [CreateAudioGraphResult](#) gives access to the created audio graph or provides an error value if audio graph creation fails.

C#

```
AudioGraph audioGraph;
```

C#

```
private async Task InitAudioGraph()
{
    AudioGraphSettings settings = new
    AudioGraphSettings(Windows.Media.Render.AudioRenderCategory.Media);

    CreateAudioGraphResult result = await AudioGraph.CreateAsync(settings);
    if (result.Status != AudioGraphCreationStatus.Success)
    {
        ShowErrorMessage("AudioGraph creation error: " +
        result.Status.ToString());
    }

    audioGraph = result.Graph;
```

```
}
```

- All audio node types are created by using the Create* methods of the **AudioGraph** class.
- The **AudioGraph.Start** method causes the audio graph to start processing audio data. The **AudioGraph.Stop** method stops audio processing. Each node in the graph can be started and stopped independently while the graph is running, but no nodes are active when the graph is stopped. **ResetAllNodes** causes all nodes in the graph to discard any data currently in their audio buffers.
- The **QuantumStarted** event occurs when the graph is starting the processing of a new quantum of audio data. The **QuantumProcessed** event occurs when the processing of a quantum is completed.
- The only **AudioGraphSettings** property that is required is **AudioRenderCategory**. Specifying this value allows the system to optimize the audio pipeline for the specified category.
- The quantum size of the audio graph determines the number of samples that are processed at one time. By default, the quantum size is 10 ms based at the default sample rate. If you specify a custom quantum size by setting the **DesiredSamplesPerQuantum** property, you must also set the **QuantumSizeSelectionMode** property to **ClosestToDesired** or the supplied value is ignored. If this value is used, the system will choose a quantum size as close as possible to the one you specify. To determine the actual quantum size, check the **SamplesPerQuantum** of the **AudioGraph** after it has been created.
- If you only plan to use the audio graph with files and don't plan to output to an audio device, it is recommended that you use the default quantum size by not setting the **DesiredSamplesPerQuantum** property.
- The **DesiredRenderDeviceAudioProcessing** property determines the amount of processing the primary render device performs on the output of the audio graph. The **Default** setting allows the system to use the default audio processing for the specified audio render category. This processing can significantly improve the sound of audio on some devices, particularly mobile devices with small speakers. The **Raw** setting can improve performance by minimizing the amount of signal processing performed, but can result in inferior sound quality on some devices.
- If the **QuantumSizeSelectionMode** is set to **LowestLatency**, the audio graph will automatically use **Raw** for **DesiredRenderDeviceAudioProcessing**.

- Starting with Windows 10, version 1803, you can set the [AudioGraphSettings.MaxPlaybackSpeedFactor](#) property to set a maximum value used for the [AudioFileInputNode.PlaybackSpeedFactor](#), [AudioFrameInputNode.PlaybackSpeedFactor](#), and [MediaSourceInputNode.PlaybackSpeedFactor](#) properties. When an audio graph supports a playback speed factor greater than 1, the system must allocate additional memory in order to maintain a sufficient buffer of audio data. For this reason, setting **MaxPlaybackSpeedFactor** to the lowest value required by your app will reduce the memory consumption of your app. If your app will only play back content at normal speed, it is recommended that you set **MaxPlaybackSpeedFactor** to 1.
- The [EncodingProperties](#) determines the audio format used by the graph. Only 32-bit float formats are supported.
- The [PrimaryRenderDevice](#) sets the primary render device for the audio graph. If you don't set this, the default system device is used. The primary render device is used to calculate the quantum sizes for other nodes in the graph. If there are no audio render devices present on the system, audio graph creation will fail.

You can let the audio graph use the default audio render device or use the [Windows.Devices.Enumeration.DeviceInformation](#) class to get a list of the system's available audio render devices by calling [FindAllAsync](#) and passing in the audio render device selector returned by [Windows.Media.Devices.MediaDevice.GetAudioRenderSelector](#). You can choose one of the returned **DeviceInformation** objects programmatically or show UI to allow the user to select a device and then use it to set the [PrimaryRenderDevice](#) property.

C#

```
Windows.Devices.Enumeration.DeviceInformationCollection devices =
    await
Windows.Devices.Enumeration.DeviceInformation.FindAllAsync(Windows.Media.Devices.MediaDevice.GetAudioRenderSelector());

// Show UI to allow the user to select a device
Windows.Devices.Enumeration.DeviceInformation selectedDevice =
    ShowMyDeviceSelectionUI(devices);

settings.PrimaryRenderDevice = selectedDevice;
```

Device input node

A device input node feeds audio into the graph from an audio capture device connected to the system, such as a microphone. Create a [DeviceInputNode](#) object that uses the system's default audio capture device by calling [CreateDeviceInputNodeAsync](#). Provide an [AudioRenderCategory](#) to allow the system to optimize the audio pipeline for the specified category.

C#

```
AudioDeviceInputNode deviceInputNode;
```

C#

```
private async Task CreateDeviceInputNode()
{
    // Create a device output node
    CreateAudioDeviceInputNodeResult result = await
audioGraph.CreateDeviceInputNodeAsync(Windows.Media.Capture.MediaCategory.Me
dia);

    if (result.Status != AudioDeviceNodeCreationStatus.Success)
    {
        // Cannot create device output node
        ShowErrorMessage(result.Status.ToString());
        return;
    }

    deviceInputNode = result.DeviceInputNode;
}
```

If you want to specify a specific audio capture device for the device input node, you can use the [Windows.Devices.Enumeration.DeviceInformation](#) class to get a list of the system's available audio capture devices by calling [FindAllAsync](#) and passing in the audio render device selector returned by [Windows.Media.Devices.MediaDevice.GetAudioCaptureSelector](#). You can choose one of the returned **DeviceInformation** objects programmatically or show UI to allow the user to select a device and then pass it into [CreateDeviceInputNodeAsync](#).

C#

```
Windows.Devices.Enumeration.DeviceInformationCollection devices =
    await
Windows.Devices.Enumeration.DeviceInformation.FindAllAsync(Windows.Media.Dev
ices.MediaDevice.GetAudioCaptureSelector());

// Show UI to allow the user to select a device
Windows.Devices.Enumeration.DeviceInformation selectedDevice =
ShowMyDeviceSelectionUI(devices);
```

```
CreateAudioDeviceInputNodeResult result =  
    await  
    audioGraph.CreateDeviceInputNodeAsync(Windows.Media.Capture.MediaCategory.Me  
dia, audioGraph.EncodingProperties, selectedDevice);
```

Device output node

A device output node pushes audio from the graph to an audio render device, such as speakers or a headset. Create a [DeviceOutputNode](#) by calling [CreateDeviceOutputNodeAsync](#). The output node uses the [PrimaryRenderDevice](#) of the audio graph.

C#

```
AudioDeviceOutputNode deviceOutputNode;
```

C#

```
private async Task CreateDeviceOutputNode()  
{  
    // Create a device output node  
    CreateAudioDeviceOutputNodeResult result = await  
    audioGraph.CreateDeviceOutputNodeAsync();  
  
    if (result.Status != AudioDeviceNodeCreationStatus.Success)  
    {  
        // Cannot create device output node  
        ShowErrorMessage(result.Status.ToString());  
        return;  
    }  
  
    deviceOutputNode = result.DeviceOutputNode;  
}
```

File input node

A file input node allows you to feed data from an audio file into the graph. Create an [AudioFileInputNode](#) by calling [CreateFileInputNodeAsync](#).

C#

```
AudioFileInputNode fileInputNode;
```

C#


```

private async Task CreateFileInputNode()
{
    if (audioGraph == null)
        return;

    FileOpenPicker filePicker = new FileOpenPicker();
    filePicker.SuggestedStartLocation = PickerLocationId.MusicLibrary;
    filePicker.FileTypeFilter.Add(".mp3");
    filePicker.FileTypeFilter.Add(".wav");
    filePicker.FileTypeFilter.Add(".wma");
    filePicker.FileTypeFilter.Add(".m4a");
    filePicker.ViewMode = PickerViewMode.Thumbnail;
    StorageFile file = await filePicker.PickSingleFileAsync();

    // File can be null if cancel is hit in the file picker
    if (file == null)
    {
        return;
    }
    CreateAudioFileInputNodeResult result = await
audioGraph.CreateFileInputNodeAsync(file);

    if (result.Status != AudioFileNodeCreationStatus.Success)
    {
        ShowErrorMessage(result.Status.ToString());
    }

    fileInputNode = result.FileInputNode;
}

```

- File input nodes support the following file formats: mp3, wav, wma, m4a.
- Set the **StartTime** property to specify the time offset into the file where playback should begin. If this property is null, the beginning of the file is used. Set the **EndTime** property to specify the time offset into the file where playback should end. If this property is null, the end of the file is used. The start time value must be lower than the end time value, and the end time value must be less than or equal to the duration of the audio file, which can be determined by checking the **Duration** property value.
- Seek to a position in the audio file by calling **Seek** and specifying the time offset into the file to which the playback position should be moved. The specified value must be within the **StartTime** and **EndTime** range. Get the current playback position of the node with the read-only **Position** property.
- Enable looping of the audio file by setting the **LoopCount** property. When non-null, this value indicates the number of times the file will be played in after the initial playback. So, for example, setting **LoopCount** to 1 will cause the file to be played 2 times in total, and setting it to 5 will cause the file to be played 6 times in

total. Setting **LoopCount** to null causes the file to be looped indefinitely. To stop looping, set the value to 0.

- Adjust the speed at which the audio file is played back by setting the **PlaybackSpeedFactor**. A value of 1 indicates the original speed of the file, .5 is half-speed, and 2 is double speed.

MediaSource input node

The **MediaSource** class provides a common way to reference media from different sources and exposes a common model for accessing media data regardless of the underlying media format which could be a file on disk, a stream, or an adaptive streaming network source. A **MediaSourceAudioInputNode** node lets you direct audio data from a **MediaSource** into the audio graph. Create a **MediaSourceAudioInputNode** by calling **CreateMediaSourceAudioInputNodeAsync**, passing in a **MediaSource** object representing the content you wish to play. A

CreateMediaSourceAudioInputNodeResult is returned which you can use to determine the status of the operation by checking the **Status** property. If the status is **Success**, you can get the created **MediaSourceAudioInputNode** by accessing the **Node** property. The following example shows the creation of a node from an **AdaptiveMediaSource** object representing content streaming over the network. For more information on working with **MediaSource**, see [Media items, playlists, and tracks](#). For more information on streaming media content over the internet, see [Adaptive streaming](#).

C#

```
MediaSourceAudioInputNode mediaSourceInputNode;
```

C#

```
private async Task CreateMediaSourceInputNode(System.Uri contentUri)
{
    if (audioGraph == null)
        return;

    var adaptiveMediaSourceResult = await
        AdaptiveMediaSource.CreateFromUriAsync(contentUri);
    if(adaptiveMediaSourceResult.Status !=
        AdaptiveMediaSourceCreationStatus.Success)
    {
        Debug.WriteLine("Failed to create AdaptiveMediaSource");
        return;
    }

    var mediaSource =
        MediaSource.CreateFromAdaptiveMediaSource(adaptiveMediaSourceResult.MediaSou
```

```

    rce);
    CreateMediaSourceAudioInputNodeResult mediaSourceAudioInputNodeResult =
        await audioGraph.CreateMediaSourceAudioInputNodeAsync(mediaSource);

    if (mediaSourceAudioInputNodeResult.Status !=
        MediaSourceAudioInputNodeCreationStatus.Success)
    {
        switch (mediaSourceAudioInputNodeResult.Status)
        {
            case MediaSourceAudioInputNodeCreationStatus.FormatNotSupported:
                Debug.WriteLine("The MediaSource uses an unsupported
format");
                break;
            case MediaSourceAudioInputNodeCreationStatus.NetworkError:
                Debug.WriteLine("The MediaSource requires a network
connection and a network-related error occurred");
                break;
            case MediaSourceAudioInputNodeCreationStatus.UnknownFailure:
            default:
                Debug.WriteLine("An unknown error occurred while opening the
MediaSource");
                break;
        }
        return;
    }

    mediaSourceInputNode = mediaSourceAudioInputNodeResult.Node;
}

```

To receive a notification when playback has reached the end of the **MediaSource** content, register a handler for the **MediaSourceCompleted** event.

C#

```

mediaSourceInputNode.MediaSourceCompleted +=
    MediaSourceInputNode_MediaSourceCompleted;

```

C#

```

private void
MediaSourceInputNode_MediaSourceCompleted(MediaSourceAudioInputNode sender,
object args)
{
    audioGraph.Stop();
}

```

While playing a file from disk is likely to always complete successfully, media streamed from a network source may fail during playback due to a change in network connection or other issues that are outside the control of the audio graph. If a **MediaSource**

becomes unplayable during playback, the audio graph will raise the [UnrecoverableErrorOccurred](#) event. You can use the handler for this event to stop and dispose of the audio graph and then reinitialize your graph.

C#

```
audioGraph.UnrecoverableErrorOccurred +=  
AudioGraph_UnrecoverableErrorOccurred;
```

C#

```
private void AudioGraph_UnrecoverableErrorOccurred(AudioGraph sender,  
AudioGraphUnrecoverableErrorOccurredEventArgs args)  
{  
    if (sender == audioGraph && args.Error !=  
AudioGraphUnrecoverableError.None)  
    {  
        Debug.WriteLine("The audio graph encountered an unrecoverable  
error.");  
        audioGraph.Stop();  
        audioGraph.Dispose();  
        InitAudioGraph();  
    }  
}
```

File output node

A file output node lets you direct audio data from the graph into an audio file. Create an [AudioFileOutputNode](#) by calling [CreateFileOutputNodeAsync](#).

C#

```
AudioFileOutputNode fileOutputNode;
```

C#

```
private async Task CreateFileOutputNode()  
{  
    FileSavePicker saveFilePicker = new FileSavePicker();  
    saveFilePicker.FileTypeChoices.Add("Pulse Code Modulation", new  
List<string>() { ".wav" });  
    saveFilePicker.FileTypeChoices.Add("Windows Media Audio", new  
List<string>() { ".wma" });  
    saveFilePicker.FileTypeChoices.Add("MPEG Audio Layer-3", new  
List<string>() { ".mp3" });  
    saveFilePicker.SuggestedFileName = "New Audio Track";  
    StorageFile file = await saveFilePicker.PickSaveFileAsync();
```

```

// File can be null if cancel is hit in the file picker
if (file == null)
{
    return;
}

Windows.Media.MediaProperties.MediaEncodingProfile mediaEncodingProfile;
switch (file.FileType.ToString().ToLowerInvariant())
{
    case ".wma":
        mediaEncodingProfile =
MediaEncodingProfile.CreateWma(AudioEncodingQuality.High);
        break;
    case ".mp3":
        mediaEncodingProfile =
MediaEncodingProfile.CreateMp3(AudioEncodingQuality.High);
        break;
    case ".wav":
        mediaEncodingProfile =
MediaEncodingProfile.CreateWav(AudioEncodingQuality.High);
        break;
    default:
        throw new ArgumentException();
}

// Operate node at the graph format, but save file at the specified
format
CreateAudioFileOutputNodeResult result = await
audioGraph.CreateFileOutputNodeAsync(file, mediaEncodingProfile);

if (result.Status != AudioFileNodeCreationStatus.Success)
{
    // FileOutputNode creation failed
    ShowErrorMessage(result.Status.ToString());
    return;
}

fileOutputNode = result.FileOutputNode;
}

```

- File output nodes support the following file formats: mp3, wav, wma, m4a.
- You must call [AudioFileOutputNode.Stop](#) to stop the node's processing before calling [AudioFileOutputNode.FinalizeAsync](#) or an exception will be thrown.

Audio frame input node

An audio frame input node allows you to push audio data that you generate in your own code into the audio graph. This enables scenarios like creating a custom software

synthesizer. Create an [AudioFrameInputNode](#) by calling [CreateFrameInputNode](#).

C#

```
AudioFrameInputNode frameInputNode;
```

C#

```
private void CreateFrameInputNode()
{
    // Create the FrameInputNode at the same format as the graph, except
    // explicitly set mono.
    AudioEncodingProperties nodeEncodingProperties =
    audioGraph.EncodingProperties;
    nodeEncodingProperties.ChannelCount = 1;
    frameInputNode =
    audioGraph.CreateFrameInputNode(nodeEncodingProperties);

    // Initialize the Frame Input Node in the stopped state
    frameInputNode.Stop();

    // Hook up an event handler so we can start generating samples when
    // needed
    // This event is triggered when the node is required to provide data
    frameInputNode.QuantumStarted += node_QuantumStarted;
}
```

The [FrameInputNode.QuantumStarted](#) event is raised when the audio graph is ready to begin processing the next quantum of audio data. You supply your custom generated audio data from within the handler to this event.

C#

```
private void node_QuantumStarted(AudioFrameInputNode sender,
    FrameInputNodeQuantumStartedEventArgs args)
{
    // GenerateAudioData can provide PCM audio data by directly synthesizing
    // it or reading from a file.
    // Need to know how many samples are required. In this case, the node is
    // running at the same rate as the rest of the graph
    // For minimum latency, only provide the required amount of samples.
    // Extra samples will introduce additional latency.
    uint numSamplesNeeded = (uint)args.RequiredSamples;

    if (numSamplesNeeded != 0)
    {
        AudioFrame audioData = GenerateAudioData(numSamplesNeeded);
        frameInputNode.AddFrame(audioData);
    }
}
```

```
}  
}
```

- The [FrameInputNodeQuantumStartedEventArgs](#) object passed into the **QuantumStarted** event handler exposes the [RequiredSamples](#) property that indicates how many samples the audio graph needs to fill up the quantum to be processed.
- Call [AudioFrameInputNode.AddFrame](#) to pass an [AudioFrame](#) object filled with audio data into the graph.
- A new set of APIs for using **MediaFrameReader** with audio data were introduced in Windows 10, version 1803. These APIs allow you to obtain **AudioFrame** objects from a media frame source, which can be passed into a **FrameInputNode** using the **AddFrame** method. For more information, see [Process audio frames with MediaFrameReader](#).
- An example implementation of the **GenerateAudioData** helper method is shown below.

To populate an [AudioFrame](#) with audio data, you must get access to the underlying memory buffer of the audio frame. To do this you must initialize the **IMemoryBufferByteAccess** COM interface by adding the following code within your namespace.

C#

```
[ComImport]  
[Guid("5B0D3235-4DBA-4D44-865E-8F1D0E4FD04D")]  
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]  
unsafe interface IMemoryBufferByteAccess  
{  
    void GetBuffer(out byte* buffer, out uint capacity);  
}
```

The following code shows an example implementation of a **GenerateAudioData** helper method that creates an [AudioFrame](#) and populates it with audio data.

C#

```
private double audioWaveTheta = 0;  
  
unsafe private AudioFrame GenerateAudioData(uint samples)  
{  
    // Buffer size is (number of samples) * (size of each sample)  
    // We choose to generate single channel (mono) audio. For multi-channel,  
    multiply by number of channels  
    uint bufferSize = samples * sizeof(float);  
    AudioFrame frame = new Windows.Media.AudioFrame(bufferSize);
```

```

        using (AudioBuffer buffer =
frame.LockBuffer(AudioBufferAccessMode.Write))
        using (IMemoryBufferReference reference = buffer.CreateReference())
        {
            byte* dataInBytes;
            uint capacityInBytes;
            float* dataInFloat;

            // Get the buffer from the AudioFrame
            ((IMemoryBufferByteAccess)reference).GetBuffer(out dataInBytes, out
capacityInBytes);

            // Cast to float since the data we are generating is float
            dataInFloat = (float*)dataInBytes;

            float freq = 1000; // choosing to generate frequency of 1kHz
            float amplitude = 0.3f;
            int sampleRate = (int)audioGraph.EncodingProperties.SampleRate;
            double sampleIncrement = (freq * (Math.PI * 2)) / sampleRate;

            // Generate a 1kHz sine wave and populate the values in the memory
buffer
            for (int i = 0; i < samples; i++)
            {
                double sinValue = amplitude * Math.Sin(audioWaveTheta);
                dataInFloat[i] = (float)sinValue;
                audioWaveTheta += sampleIncrement;
            }
        }

        return frame;
    }
}

```

- Because this method accesses the raw buffer underlying the Windows Runtime types, it must be declared using the **unsafe** keyword. You must also configure your project in Microsoft Visual Studio to allow the compilation of unsafe code by opening the project's **Properties** page, clicking the **Build** property page, and selecting the **Allow Unsafe Code** checkbox.
- Initialize a new instance of **AudioFrame**, in the **Windows.Media** namespace, by passing in the desired buffer size to the constructor. The buffer size is the number of samples multiplied by the size of each sample.
- Get the **AudioBuffer** of the audio frame by calling **LockBuffer**.
- Get an instance of the **IMemoryBufferByteAccess** COM interface from the audio buffer by calling **CreateReference**.
- Get a pointer to raw audio buffer data by calling **IMemoryBufferByteAccess.GetBuffer** and cast it to the sample data type of the audio data.

- Fill the buffer with data and return the [AudioFrame](#) for submission into the audio graph.

Audio frame output node

An audio frame output node allows you to receive and process audio data output from the audio graph with custom code that you create. An example scenario for this is performing signal analysis on the audio output. Create an [AudioFrameOutputNode](#) by calling [CreateFrameOutputNode](#).

C#

```
AudioFrameOutputNode frameOutputNode;
```

C#

```
private void CreateFrameOutputNode()
{
    frameOutputNode = audioGraph.CreateFrameOutputNode();
    audioGraph.QuantumStarted += AudioGraph_QuantumStarted;
}
```

The [AudioGraph.QuantumStarted](#) event is raised when the audio graph has begins processing a quantum of audio data. You can access the audio data from within the handler for this event.

ⓘ Note

If you want to retrieve audio frames on a regular cadence, synchronized with the audio graph, call [AudioFrameOutputNode.GetFrame](#) from within the synchronous **QuantumStarted** event handler. The **QuantumProcessed** event is raised asynchronously after the audio engine has completed audio processing, which means its cadence may be irregular. Therefore you should not use the **QuantumProcessed** event for synchronized processing of audio frame data.

C#

```
private void AudioGraph_QuantumStarted(AudioGraph sender, object args)
{
    AudioFrame frame = frameOutputNode.GetFrame();
    ProcessFrameOutput(frame);
}
```

```
}
```

- Call [GetFrame](#) to get an [AudioFrame](#) object filled with audio data from the graph.
- An example implementation of the [ProcessFrameOutput](#) helper method is shown below.

C#

```
unsafe private void ProcessFrameOutput(AudioFrame frame)
{
    using (AudioBuffer buffer =
frame.LockBuffer(AudioBufferAccessMode.Write))
        using (IMemoryBufferReference reference = buffer.CreateReference())
        {
            byte* dataInBytes;
            uint capacityInBytes;
            float* dataInFloat;

            // Get the buffer from the AudioFrame
            ((IMemoryBufferByteAccess)reference).GetBuffer(out dataInBytes, out
capacityInBytes);

            dataInFloat = (float*)dataInBytes;
        }
}
```

- Like the audio frame input node example above, you will need to declare the [IMemoryBufferByteAccess](#) COM interface and configure your project to allow unsafe code in order to access the underlying audio buffer.
- Get the [AudioBuffer](#) of the audio frame by calling [LockBuffer](#).
- Get an instance of the [IMemoryBufferByteAccess](#) COM interface from the audio buffer by calling [CreateReference](#).
- Get a pointer to raw audio buffer data by calling [IMemoryBufferByteAccess.GetBuffer](#) and cast it to the sample data type of the audio data.

Node connections and submix nodes

All input nodes types expose the [AddOutgoingConnection](#) method that routes the audio produced by the node to the node that is passed into the method. The following example connects an [AudioFileInputNode](#) to an [AudioDeviceOutputNode](#), which is a simple setup for playing an audio file on the device's speaker.

C#

```
fileInputNode.AddOutgoingConnection(deviceOutputNode);
```

You can create more than one connection from an input node to other nodes. The following example adds another connection from the [AudioFileInputNode](#) to an [AudioFileOutputNode](#). Now, the audio from the audio file is played to the device's speaker and is also written out to an audio file.

C#

```
fileInputNode.AddOutgoingConnection(fileOutputNode);
```

Output nodes can also receive more than one connection from other nodes. In the following example a connection is made from a [AudioDeviceInputNode](#) to the [AudioDeviceOutput](#) node. Because the output node has connections from the file input node and the device input node, the output will contain a mix of audio from both sources. **AddOutgoingConnection** provides an overload that lets you specify a gain value for the signal passing through the connection.

C#

```
deviceInputNode.AddOutgoingConnection(deviceOutputNode, .5);
```

Although output nodes can accept connections from multiple nodes, you may want to create an intermediate mix of signals from one or more nodes before passing the mix to an output. For example, you may want to set the level or apply effects to a subset of the audio signals in a graph. To do this, use the [AudioSubmixNode](#). You can connect to a submix node from one or more input nodes or other submix nodes. In the following example, a new submix node is created with [AudioGraph.CreateSubmixNode](#). Then, connections are added from a file input node and a frame output node to the submix node. Finally, the submix node is connected to a file output node.

C#

```
private void CreateSubmixNode()
{
    AudioSubmixNode submixNode = audioGraph.CreateSubmixNode();
    fileInputNode.AddOutgoingConnection(submixNode);
    frameInputNode.AddOutgoingConnection(submixNode);
    submixNode.AddOutgoingConnection(fileOutputNode);
}
```

Starting and stopping audio graph nodes

When [AudioGraph.Start](#) is called, the audio graph begins processing audio data. Every node type provides **Start** and **Stop** methods that cause the individual node to start or stop processing data. When [AudioGraph.Stop](#) is called, all audio processing in the all nodes is stopped regardless of the state of individual nodes, but the state of each node can be set while the audio graph is stopped. For example, you could call **Stop** on an individual node while the graph is stopped and then call [AudioGraph.Start](#), and the individual node will remain in the stopped state.

All node types expose the **ConsumeInput** property that, when set to false, allows the node to continue audio processing but stops it from consuming any audio data being input from other nodes.

All node types expose the **Reset** method that causes the node to discard any audio data currently in its buffer.

Adding audio effects

The audio graph API allows you to add audio effects to every type of node in a graph. Output nodes, input nodes, and submix nodes can each have an unlimited number of audio effects, limited only by the capabilities of the hardware. The following example demonstrates adding the built-in echo effect to a submix node.

C#

```
EchoEffectDefinition echoEffect = new EchoEffectDefinition(audioGraph);
echoEffect.Delay = 1000.0;
echoEffect.Feedback = .2;
echoEffect.WetDryMix = .5;

submixNode.EffectDefinitions.Add(echoEffect);
```

- All audio effects implement [IAudioEffectDefinition](#). Every node exposes an **EffectDefinitions** property representing the list of effects applied to that node. Add an effect by adding it's definition object to the list.
- There are several effect definition classes that are provided in the **Windows.Media.Audio** namespace. These include:
 - [EchoEffectDefinition](#)
 - [EqualizerEffectDefinition](#)
 - [LimiterEffectDefinition](#)
 - [ReverbEffectDefinition](#)

- You can create your own audio effects that implement [IAudioEffectDefinition](#) and apply them to any node in an audio graph.
- Every node type exposes a **DisableEffectsByDefinition** method that disables all effects in the node's **EffectDefinitions** list that were added using the specified definition. **EnableEffectsByDefinition** enables the effects with the specified definition.

Spatial audio

Starting with Windows 10, version 1607, **AudioGraph** supports spatial audio, which allows you to specify the location in 3D space from which audio from any input or submix node is emitted. You can also specify a shape and direction in which audio is emitted, a velocity that will be used to Doppler shift the node's audio, and define a decay model that describes how the audio is attenuated with distance.

To create an emitter, you can first create a shape in which the sound is projected from the emitter, which can be a cone or omnidirectional. The [AudioNodeEmitterShape](#) class provides static methods for creating each of these shapes. Next, create a decay model. This defines how the volume of the audio from the emitter decreases as the distance from the listener increases. The [CreateNatural](#) method creates a decay model that emulates the natural decay of sound using a distance squared falloff model. Finally, create an [AudioNodeEmitterSettings](#) object. Currently, this object is only used to enable and disable velocity-based Doppler attenuation of the emitter's audio. Call the [AudioNodeEmitter](#) constructor, passing in the initialization objects you just created. By default, the emitter is placed at the origin, but you can set the position of the emitter with the [Position](#) property.

ⓘ Note

Audio node emitters can only process audio that is formatted in mono with a sample rate of 48kHz. Attempting to use stereo audio or audio with a different sample rate will result in an exception.

You assign the emitter to an audio node when you create it by using the overloaded creation method for the type of node you want. In this example, [CreateFileInputNodeAsync](#) is used to create a file input node from a specified file and the [AudioNodeEmitter](#) object you want to associate with the node.

C#

```

var emitterShape = AudioNodeEmitterShape.CreateOmnidirectional();
var decayModel = AudioNodeEmitterDecayModel.CreateNatural(.1, 1, 10, 100);
var settings = AudioNodeEmitterSettings.None;

var emitter = new AudioNodeEmitter(emitterShape, decayModel, settings);
emitter.Position = new System.Numerics.Vector3(10, 0, 5);

CreateAudioFileInputNodeResult result = await
audioGraph.CreateFileInputNodeAsync(file, emitter);

if (result.Status != AudioFileNodeCreationStatus.Success)
{
    ShowErrorMessage(result.Status.ToString());
}

fileInputNode = result.FileInputNode;

```

The **AudioDeviceOutputNode** that outputs audio from the graph to the user has a listener object, accessed with the **Listener** property, which represents the location, orientation, and velocity of the user in the 3D space. The positions of all of the emitters in the graph are relative to the position and orientation of the listener object. By default, the listener is located at the origin (0,0,0) facing forward along the Z axis, but you can set its position and orientation with the **Position** and **Orientation** properties.

C#

```

deviceOutputNode.Listener.Position = new System.Numerics.Vector3(100, 0, 0);
deviceOutputNode.Listener.Orientation =
System.Numerics.Quaternion.CreateFromYawPitchRoll(0, (float)Math.PI, 0);

```

You can update the location, velocity, and direction of emitters at runtime to simulate the movement of an audio source through 3D space.

C#

```

var emitter = fileInputNode.Emitter;
emitter.Position = newObjectPosition;
emitter.DopplerVelocity = newObjectPosition - oldObjectPosition;

```

You can also update the location, velocity, and orientation of the listener object at runtime to simulate the movement of the user through 3D space.

C#

```

deviceOutputNode.Listener.Position = newUserPosition;

```

By default, spatial audio is calculated using Microsoft's head-relative transfer function (HRTF) algorithm to attenuate the audio based on its shape, velocity, and position relative to the listener. You can set the [SpatialAudioModel](#) property to **FoldDown** to use a simple stereo mix method of simulating spatial audio that is less accurate but requires less CPU and memory resources.

See also

- [Media playback](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

MIDI

Article • 10/20/2022

This article shows you how to enumerate MIDI (Musical Instrument Digital Interface) devices and send and receive MIDI messages from a Universal Windows app. Windows 10 supports MIDI over USB (class-compliant and most proprietary drivers), MIDI over Bluetooth LE (Windows 10 Anniversary Edition and later), and through freely-available third-party products, MIDI over Ethernet and routed MIDI.

Enumerate MIDI devices

Before enumerating and using MIDI devices, add the following namespaces to your project.

C#

```
using Windows.Devices.Enumeration;  
using Windows.Devices.Midi;  
using System.Threading.Tasks;
```

Add a [ListBox](#) control to your XAML page that will allow the user to select one of the MIDI input devices attached to the system. Add another one to list the MIDI output devices.

XML

```
<ListBox x:Name="midiInPortListBox"  
SelectionChanged="midiInPortListBox_SelectionChanged"/>  
<ListBox x:Name="midiOutPortListBox"  
SelectionChanged="midiOutPortListBox_SelectionChanged"/>
```

The [FindAllAsync](#) method [DeviceInformation](#) class is used to enumerate many different types of devices that are recognized by Windows. To specify that you only want the method to find MIDI input devices, use the selector string returned by [MidiInPort.GetDeviceSelector](#). [FindAllAsync](#) returns a [DeviceInformationCollection](#) that contains a [DeviceInformation](#) for each MIDI input device registered with the system. If the returned collection contains no items, then there are no available MIDI input devices. If there are items in the collection, loop through the [DeviceInformation](#) objects and add the name of each device to the MIDI input device [ListBox](#).

C#


```

private async Task EnumerateMidiInputDevices()
{
    // Find all input MIDI devices
    string midiInputQueryString = MidiInPort.GetDeviceSelector();
    DeviceInformationCollection midiInputDevices = await
DeviceInformation.FindAllAsync(midiInputQueryString);

    midiInPortListBox.Items.Clear();

    // Return if no external devices are connected
    if (midiInputDevices.Count == 0)
    {
        this.midiInPortListBox.Items.Add("No MIDI input devices found!");
        this.midiInPortListBox.IsEnabled = false;
        return;
    }

    // Else, add each connected input device to the list
    foreach (DeviceInformation deviceInfo in midiInputDevices)
    {
        this.midiInPortListBox.Items.Add(deviceInfo.Name);
    }
    this.midiInPortListBox.IsEnabled = true;
}

```

Enumerating MIDI output devices works the exact same way as enumerating input devices, except that you should specify the selector string returned by [MidiOutPort.GetDeviceSelector](#) when calling `FindAllAsync`.

C#

```

private async Task EnumerateMidiOutputDevices()
{
    // Find all output MIDI devices
    string midiOutputQueryString = MidiOutPort.GetDeviceSelector();
    DeviceInformationCollection midiOutputDevices = await
DeviceInformation.FindAllAsync(midiOutputQueryString);

    midiOutPortListBox.Items.Clear();

    // Return if no external devices are connected
    if (midiOutputDevices.Count == 0)
    {
        this.midiOutPortListBox.Items.Add("No MIDI output devices found!");
        this.midiOutPortListBox.IsEnabled = false;
        return;
    }

    // Else, add each connected input device to the list
    foreach (DeviceInformation deviceInfo in midiOutputDevices)
    {

```

```
        this.midiOutPortListBox.Items.Add(deviceInfo.Name);  
    }  
    this.midiOutPortListBox.IsEnabled = true;  
}
```

Create a device watcher helper class

The [Windows.Devices.Enumeration](#) namespace provides the [DeviceWatcher](#) which can notify your app if devices are added or removed from the system, or if the information for a device is updated. Since MIDI-enabled apps typically are interested in both input and output devices, this example creates a helper class that implements the [DeviceWatcher](#) pattern, so that the same code can be used for both MIDI input and MIDI output devices, without the need for duplication.

Add a new class to your project to serve as your device watcher. In this example the class is named **MyMidiDeviceWatcher**. The rest of the code in this section is used to implement the helper class.

Add some member variables to the class:

- A [DeviceWatcher](#) object that will monitor for device changes.
- A device selector string that will contain the MIDI in port selector string for one instance and the MIDI out port selector string for another instance.
- A [ListBox](#) control that will be populated with the names of the available devices.
- A [CoreDispatcher](#) that is required to update the UI from a thread other than the UI thread.

C#

```
DeviceWatcher deviceWatcher;  
string deviceSelectorString;  
ListBox deviceListBox;  
CoreDispatcher coreDispatcher;
```

Add a [DeviceInformationCollection](#) property that is used to access the current list of devices from outside the helper class.

C#

```
public DeviceInformationCollection DeviceInformationCollection { get; set; }
```

In class constructor, the caller passes in the MIDI device selector string, the **ListBox** for listing the devices, and the **Dispatcher** needed to update the UI.

Call [DeviceInformation.CreateWatcher](#) to create a new instance of the **DeviceWatcher** class, passing in the MIDI device selector string.

Register handlers for the watcher's event handlers.

C#

```
public MyMidiDeviceWatcher(string midiDeviceSelectorString, ListBox
midiDeviceListBox, CoreDispatcher dispatcher)
{
    deviceListBox = midiDeviceListBox;
    coreDispatcher = dispatcher;

    deviceSelectorString = midiDeviceSelectorString;

    deviceWatcher = DeviceInformation.CreateWatcher(deviceSelectorString);
    deviceWatcher.Added += DeviceWatcher_Added;
    deviceWatcher.Removed += DeviceWatcher_Removed;
    deviceWatcher.Updated += DeviceWatcher_Updated;
    deviceWatcher.EnumerationCompleted +=
DeviceWatcher_EnumerationCompleted;
}
```

The **DeviceWatcher** has the following events:

- [Added](#) - Raised when a new device is added to the system.
- [Removed](#) - Raised when a device is removed from the system.
- [Updated](#) - Raised when the information associated with an existing device is updated.
- [EnumerationCompleted](#) - Raised when the watcher has completed its enumeration of the requested device type.

In the event handler for each of these events, a helper method, **UpdateDevices**, is called to update the **ListBox** with the current list of devices. Because **UpdateDevices** updates UI elements and these event handlers are not called on the UI thread, each call must be wrapped in a call to [RunAsync](#), which causes the specified code to be run on the UI thread.

C#

```
private async void DeviceWatcher_Removed(DeviceWatcher sender,
DeviceInformationUpdate args)
{
    await coreDispatcher.RunAsync(CoreDispatcherPriority.High, () =>
    {
        // Update the device list
        UpdateDevices();
    });
}
```

```

}

private async void DeviceWatcher_Added(DeviceWatcher sender,
DeviceInformation args)
{
    await coreDispatcher.RunAsync(CoreDispatcherPriority.High, () =>
    {
        // Update the device list
        UpdateDevices();
    });
}

private async void DeviceWatcher_EnumerationCompleted(DeviceWatcher sender,
object args)
{
    await coreDispatcher.RunAsync(CoreDispatcherPriority.High, () =>
    {
        // Update the device list
        UpdateDevices();
    });
}

private async void DeviceWatcher_Updated(DeviceWatcher sender,
DeviceInformationUpdate args)
{
    await coreDispatcher.RunAsync(CoreDispatcherPriority.High, () =>
    {
        // Update the device list
        UpdateDevices();
    });
}

```

The **UpdateDevices** helper method calls [DeviceInformation.FindAllAsync](#) and updates the **ListBox** with the names of the returned devices as described previously in this article.

C#

```

private async void UpdateDevices()
{
    // Get a list of all MIDI devices
    this.DeviceInformationCollection = await
DeviceInformation.FindAllAsync(deviceSelectorString);

    deviceListBox.Items.Clear();

    if (!this.DeviceInformationCollection.Any())
    {
        deviceListBox.Items.Add("No MIDI devices found!");
    }

    foreach (var deviceInformation in this.DeviceInformationCollection)

```

```

    {
        deviceListBox.Items.Add(deviceInformation.Name);
    }
}

```

Add methods to start the watcher, using the **DeviceWatcher** object's [Start](#) method, and to stop the watcher, using the [Stop](#) method.

C#

```

public void StartWatcher()
{
    deviceWatcher.Start();
}
public void StopWatcher()
{
    deviceWatcher.Stop();
}

```

Provide a destructor to unregister the watcher event handlers and set the device watcher to null.

C#

```

~MyMidiDeviceWatcher()
{
    deviceWatcher.Added -= DeviceWatcher_Added;
    deviceWatcher.Removed -= DeviceWatcher_Removed;
    deviceWatcher.Updated -= DeviceWatcher_Updated;
    deviceWatcher EnumerationCompleted -=
DeviceWatcher_EnumerationCompleted;
    deviceWatcher = null;
}

```

Create MIDI ports to send and receive messages

In the code behind for your page, declare member variables to hold two instances of the **MyMidiDeviceWatcher** helper class, one for input devices and one for output devices.

C#

```

MyMidiDeviceWatcher inputDeviceWatcher;
MyMidiDeviceWatcher outputDeviceWatcher;

```

Create a new instance of the watcher helper classes, passing in the device selector string, the **ListBox** to be populated, and the **CoreDispatcher** object that can be accessed through the page's **Dispatcher** property. Then, call the method to start each object's **DeviceWatcher**.

Shortly after each **DeviceWatcher** is started, it will finish enumerating the current devices connected to the system and raise its **EnumerationCompleted** event, which will cause each **ListBox** to be updated with the current MIDI devices.

C#

```
inputDeviceWatcher =  
    new MyMidiDeviceWatcher(MidiInPort.GetDeviceSelector(),  
        midiInPortListBox, Dispatcher);  
  
inputDeviceWatcher.StartWatcher();  
  
outputDeviceWatcher =  
    new MyMidiDeviceWatcher(MidiOutPort.GetDeviceSelector(),  
        midiOutPortListBox, Dispatcher);  
  
outputDeviceWatcher.StartWatcher();
```

When the user selects an item in the MIDI input **ListBox**, the **SelectionChanged** event is raised. In the handler for this event, access the **DeviceInformationCollection** property of the helper class to get the current list of devices. If there are entries in the list, select the **DeviceInformation** object with the index corresponding to the **ListBox** control's **SelectedIndex**.

Create the **MidiInPort** object representing the selected input device by calling **MidiInPort.FromIdAsync**, passing in the **Id** property of the selected device.

Register a handler for the **MessageReceived** event, which is raised whenever a MIDI message is received through the specified device.

C#

```
MidiInPort midiInPort;  
IMidiOutPort midiOutPort;
```

C#

```
private async void midiInPortListBox_SelectionChanged(object sender,  
    SelectionChangedEventArgs e)  
{  
    var deviceInformationCollection =  
        inputDeviceWatcher.DeviceInformationCollection;
```

```

        if (deviceInformationCollection == null)
        {
            return;
        }

        DeviceInformation devInfo =
deviceInformationCollection[midiInPortListBox.SelectedIndex];

        if (devInfo == null)
        {
            return;
        }

        midiInPort = await MidiInPort.FromIdAsync(devInfo.Id);

        if (midiInPort == null)
        {
            System.Diagnostics.Debug.WriteLine("Unable to create MidiInPort from
input device");
            return;
        }
        midiInPort.MessageReceived += MidiInPort_MessageReceived;
    }

```

When the **MessageReceived** handler is called, the message is contained in the **Message** property of the **MidiMessageReceivedEventArgs**. The **Type** of the message object is a value from the **MidiMessageType** enumeration indicating the type of message that was received. The data of the message depends on the type of the message. This example checks to see if the message is a note on message and, if so, outputs the midi channel, note, and velocity of the message.

C#

```

private void MidiInPort_MessageReceived(MidiInPort sender,
MidiMessageReceivedEventArgs args)
{
    IMidiMessage receivedMidiMessage = args.Message;

    System.Diagnostics.Debug.WriteLine(receivedMidiMessage.Timestamp.ToString())
;

    if (receivedMidiMessage.Type == MidiMessageType.NoteOn)
    {

        System.Diagnostics.Debug.WriteLine(((MidiNoteOnMessage)receivedMidiMessage).
Channel);

        System.Diagnostics.Debug.WriteLine(((MidiNoteOnMessage)receivedMidiMessage).
Note);
    }
}

```

```

System.Diagnostics.Debug.WriteLine(((MidiNoteOnMessage)receivedMidiMessage).
Velocity);
    }
}

```

The [SelectionChanged](#) handler for the output device **ListBox** works the same as the handler for input devices, except no event handler is registered.

C#

```

private async void midiOutPortListBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    var deviceInformationCollection =
outputDeviceWatcher.DeviceInformationCollection;

    if (deviceInformationCollection == null)
    {
        return;
    }

    DeviceInformation devInfo =
deviceInformationCollection[midiOutPortListBox.SelectedIndex];

    if (devInfo == null)
    {
        return;
    }

    midiOutPort = await MidiOutPort.FromIdAsync(devInfo.Id);

    if (midiOutPort == null)
    {
        System.Diagnostics.Debug.WriteLine("Unable to create MidiOutPort
from output device");
        return;
    }
}
}

```

Once the output device is created, you can send a message by creating a new [IMidiMessage](#) for the type of message you want to send. In this example, the message is a [NoteOnMessage](#). The [SendMessage](#) method of the [IMidiOutPort](#) object is called to send the message.

C#

```

byte channel = 0;
byte note = 60;

```



```
byte velocity = 127;
IMidiMessage midiMessageToSend = new MidiNoteOnMessage(channel, note,
velocity);

midiOutPort.SendMessage(midiMessageToSend);
```

When your app is deactivated, be sure to clean up your apps resources. Unregister your event handlers and set the MIDI in port and out port objects to null. Stop the device watchers and set them to null.

```
C#

inputDeviceWatcher.StopWatcher();
inputDeviceWatcher = null;

outputDeviceWatcher.StopWatcher();
outputDeviceWatcher = null;

midiInPort.MessageReceived -= MidiInPort_MessageReceived;
midiInPort.Dispose();
midiInPort = null;

midiOutPort.Dispose();
midiOutPort = null;
```

Using the built-in Windows General MIDI synth

When you enumerate output MIDI devices using the technique described above, your app will discover a MIDI device called "Microsoft GS Wavetable Synth". This is a built-in General MIDI synthesizer that you can play from your app. However, attempting to create a MIDI output to this device will fail unless you have included the SDK extension for the built-in synth in your project.

To include the General MIDI Synth SDK extension in your app project

1. In **Solution Explorer**, under your project, right-click **References** and select **Add reference...**
2. Expand the **Universal Windows** node.
3. Select **Extensions**.
4. From the list of extensions, select **Microsoft General MIDI DLS for Universal Windows Apps**.

ⓘ **Note**

If there are multiple versions of the extension, be sure to select the version that matches the target for your app. You can see which SDK version your app is targeting on the **Application** tab of the project Properties.

Import media from a device

Article • 10/20/2022

This article describes how to import media from a device, including searching for available media sources, importing files such as videos, photos, and sidecar files, and deleting the imported files from the source device.

ⓘ Note

The code in this article was adapted from the [MediaImport UWP app sample](#) . You can clone or download this sample from the [Universal Windows app samples Git repo](#) to see the code in context or to use it as a starting point for your own app.

Create a simple media import UI

The example in this article uses a minimal UI to enable the core media import scenarios. To see how to create a more robust UI for a media import app, see the [MediaImport sample](#) . The following XAML creates a stack panel with the following controls:

- A **Button** to initiate searching for sources from which media can be imported.
- A **ComboBox** to list and select from the media import sources that are found.
- A **ListView** control to display and select from the media items from the selected import source.
- A **Button** to initiate importing media items from the selected source.
- A **Button** to initiate deleting the items that have been imported from the selected source.
- A **Button** to cancel an asynchronous media import operation.

XML

```
<StackPanel Orientation="Vertical">
    <Button x:Name="findSourcesButton" Click="findSourcesButton_Click"
Content="Find sources"/>
    <ComboBox x:Name="sourcesComboBox"
SelectionChanged="sourcesComboBox_SelectionChanged"/>
    <ListView x:Name="fileListView"
        HorizontalAlignment="Left" Margin="182,260,0,171"
        Width="715"
        SelectionMode="None"
        BorderBrush="#FF858585"
        BorderThickness="1"
        ScrollViewer.VerticalScrollBarVisibility="Visible">
```

```

<ListView.ItemTemplate>
    <DataTemplate>
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="0.05*" />
                <ColumnDefinition Width="0.20*" />
                <ColumnDefinition Width="0.75*" />
            </Grid.ColumnDefinitions>
            <CheckBox Grid.Column="0" IsChecked="{Binding
ImportableItem.IsSelected, Mode=TwoWay}" />
            <!-- Click="CheckBox_Click"/>-->
            <Image Grid.Column="1" Source="{Binding Thumbnail}"
Width="120" Height="120" Stretch="Uniform" />
            <TextBlock Grid.Column="2" Text="{Binding
ImportableItem.Name}" VerticalAlignment="Center" Margin="10,0" />
        </Grid>
    </DataTemplate>
</ListView.ItemTemplate>
</ListView>

<Button x:Name="importButton" Click="importButton_Click"
Content="Import" />
<Button x:Name="deleteButton" Click="deleteButton_Click"
Content="Delete" />
<Button x:Name="cancelButton" Click="cancelButton_Click"
Content="Cancel" />
<ProgressBar x:Name="progressBar" SmallChange="0.01" LargeChange="0.1"
Maximum="1" />

</StackPanel>

```

Set up your code-behind file

Add *using* directives to include the namespaces used by this example that are not already included in the default project template.

C#

```

using Windows.Media.Import;
using System.Threading;
using Windows.UI.Core;
using System.Text;

```

Set up task cancellation for media import operations

Because media import operations can take a long time, they are performed asynchronously using [IAsyncOperationWithProgress](#). Declare a class member variable of type [CancellationTokenSource](#) that will be used to cancel an in-progress operation if the user clicks the cancel button.

C#

```
CancellationTokenSource cts;
```

Implement a handler for the cancel button. The examples shown later in this article will initialize the **CancellationTokenSource** when an operation begins and set it to null when the operation completes. In the cancel button handler, check to see if the token is null, and if not, call [Cancel](#) to cancel the operation.

C#

```
private void cancelButton_Click(object sender, RoutedEventArgs e)
{
    if (cts != null)
    {
        cts.Cancel();
        System.Diagnostics.Debug.WriteLine("Operation canceled by the Cancel
button.");
    }
}
```

Data binding helper classes

In a typical media import scenario you show the user a list of available media items to import, there can be a large number of media files to choose from and, typically, you want to show a thumbnail for each media item. For this reason, this example uses three helper classes to incrementally load entries into the ListView control as the user scrolls down through the list.

- **IncrementalLoadingBase** class - Implements the [IList](#), [ISupportIncrementalLoading](#), and [INotifyCollectionChanged](#) to provide the base incremental loading behavior.
- **GeneratorIncrementalLoadingClass** class - Provides an implementation of the incremental loading base class.
- **ImportableItemWrapper** class - A thin wrapper around the [PhotoImportItem](#) class to add a bindable [BitmapImage](#) property for the thumbnail image for each imported item.